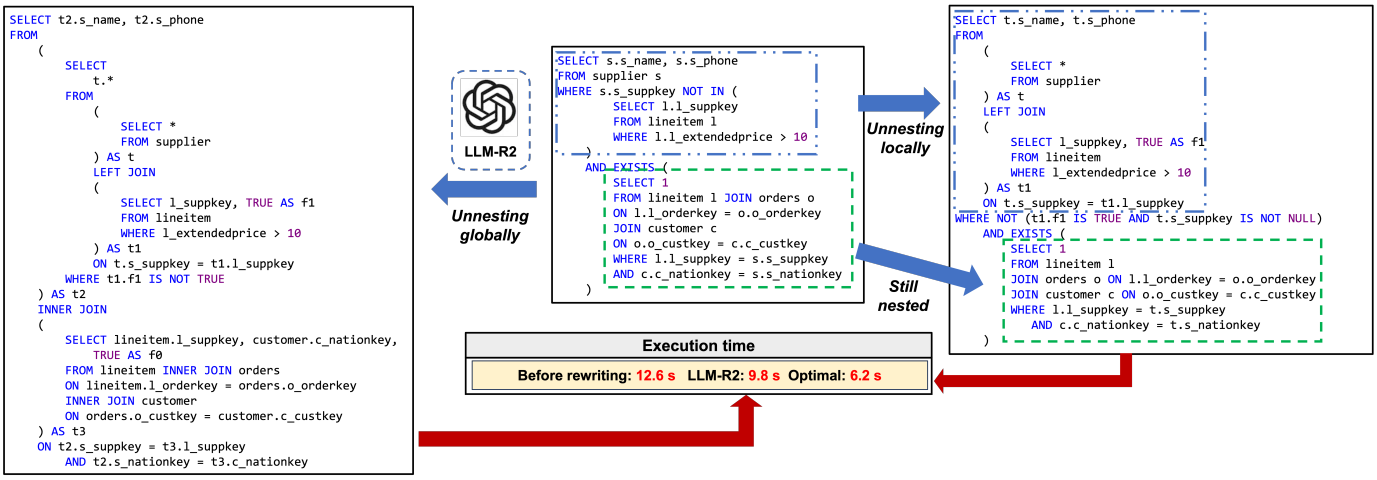

Fig. 11: Illustration of the necessity of determining whether to unnest a nested sub-query individually rather than performing unnesting for all sub-queries globally

A. One additional motivating example

As shown in 11, there are two nested sub-queries in the query in the middle box. To rewrite this query, LLM-R2 [8] would recommend an unnesting rule that would perform the unnesting operation globally. This means that *all* sub-queries in this query would be unnested into derived table expressions. However, as pointed out in [25], unnesting sub-queries does not always bring performance gains. Hence, after taking a closer look at the original query in Figure 11, we observe that merely unnesting the first sub-query while maintaining the second sub-query nested would lead to shorter execution time than that of unnesting both sub-queries (6.2 seconds VS 9.8 seconds). This thus highlights the necessity of determining whether to unnest a nested sub-query at the fine-grained level for producing a more efficient rewritten query.

B. Categorization of parent-child rules and local rules

We summarize different categories of parent-child rules in Table V. Note that in this table, some rules may belong to multiple categories since their rewriting effect may be varied in different contexts. In total, there are 27 unique parent-child rules.

Type of parent-child rules	Rules in Apache Calcite
Unnesting rule	FILTER_INTO_JOIN, SEMI_JOIN_REMOVE
Pull-up rule	UNION_TO_DISTINCT, AGGREGATE_ANY_PULL_UP_CONSTANTS, UNION_PULL_UP_CONSTANTS, AGGREGATE_UNION_AGGREGATE, AGGREGATE_UNION_TRANSPOSE
Push-down rule	FILTER_AGGREGATE_TRANSPOSE, FILTER_CORRELATE, FILTER_INTO_JOIN, FILTER_PROJECT_TRANSPOSE, FILTER_SET_OP_TRANSPOSE, FILTER_SCAN, JOIN_CONDITION_PUSH, AGGREGATE_JOIN_TRANSPOSE_EXTENDED, SORT_JOIN_TRANSPOSE, SORT_PROJECT_TRANSPOSE, SORT_UNION_TRANSPOSE, JOIN_LEFT_UNION_TRANSPOSE, JOIN_RIGHT_UNION_TRANSPOSE, FILTER_TABLE_FUNCTION_TRANSPOSE
Removal rule	UNION_PULL_UP_CONSTANTS, AGGREGATE_REMOVE, JOIN_REDUCE_EXPRESSIONS, AGGREGATE_ANY_PULL_UP_CONSTANTS, AGGREGATE_PROJECT_MERGE, UNION_MERGE, PROJECT_REDUCE_EXPRESSIONS, FILTER_REDUCE_EXPRESSIONS, FILTER_MERGE

TABLE V: Categorization of parent-child rules in Apache Calcite into the Unnesting rule, Pull-up rule, Push-down rule, and Removal rule

We also list all local rules in Table VI. In total, there are 19 rules in Apache Calcite identified as local rules.

C. Additional theoretical analysis

A. Proof Sketch to Corollary 1

Proof Sketch. According to the definition of the parent-child rules, since each of their invocations only modifies one sub-query, then the invocations of the parent-child rules across sub-queries in Q are indeed independent. Hence, we can apply the

	Rules in Apache Calcite
Local rules	PROJECT_CALC_MERGE, PROJECT_MERGE,PROJECT_REMOVE, PROJECT_TO_CALC, AGGREGATE_INSTANCE SORT_REMOVE_CONSTANT_KEYS, SORT_REMOVE, SORT_FETCH_ZERO_INSTANCE, CALC_MERGE, CALC_REMOVE, FILTER_INSTANCE, JOIN_LEFT_INSTANCE, JOIN_RIGHT_INSTANCE, PROJECT_INSTANCE, SORT_INSTANCE,UNION_INSTANCE,INTERSECT_INSTANCE,MINUS_INSTANCE, AGGREGATE_VALUES

TABLE VI: Local rules in Apache Calcite

rewriting rule types (unnesting, removal, pull-up, push-down, local) in a fixed order specified by Theorem 3 across all sub-queries simultaneously, leveraging their independence. The only adjustment is delaying the deletion of pushed-down operations in the outer query until all sub-queries are processed. This can ensure the correctness when an operation is pushed down to multiple sub-queries. □

B. Proof Sketch to Corollary 2

Proof Sketch. Although parent-child rules take effect between one query block and one of its sub-queries, their rewriting effect can be incrementally propagated throughout the entire query tree. Hence, it is crucial to determine their invocation orders across different tree layers. To correctly rewrite the query tree using parent-child rules, we first perform a top-down traversal to apply unnesting rules followed by removal rules, ensuring unnested but redundant sub-queries are eliminated early. Next, a bottom-up traversal recursively invokes pull-up rules, allowing operations from child blocks to propagate upward for further optimization. A subsequent top-down traversal applies push-down rules, enabling operations from parent blocks to descend through the tree. Finally, after all parent-child rules are executed, local rules are invoked at each query block (by Corollary 1). This ordered approach guarantees that operations are propagated correctly while unnecessary sub-queries are pruned, preserving correctness while maintaining the optimality of $\mathcal{R}$. □

D. Additional ablation studies

A. Effect of the LLMs

To examine the impact of different pretrained large language models on query rewriting performance, both the proposed BQR method and the strongest baseline, LLM-R2, are evaluated on LR-TPC-H dataset using GPT-4 and Qwen3-235B. These two methods represent different levels of dependency on LLMs for rule derivation: LLM-R2 relies entirely on the model to infer rewrite rules, whereas BQR employs LLMs only for local rule generation within its hybrid optimization framework. As shown in Table VII, both GPT-4 and Qwen3-235B achieve comparable results, indicating that current state-of-the-art pretrained LLMs generally possess sufficient capability to infer effective query rewriting rules. Moreover, the performance gap between different LLMs is smaller for BQR, reflecting its reduced sensitivity to model choice due to limited reliance on LLM-based rule inference.

TABLE VII: Effect of the LLMs

Execution time (seconds)	LR-TPC-H			
Method	Mean	Median	80th	95th
Original	174.7567	63.9999	307.2657	600.0000
LLM-R2 (GPT-4)	179.3904	43.2653	600.0000	600.0000
LLM-R2 (Qwen3)	178.6174	45.7565	600.0000	600.0000
BQR (GPT-4)	139.5337	**23.5171**	273.9349	**600.0000**
BQR (Qwen3)	**139.2376**	24.0125	**271.6807**	600.0000